Griffith UNIVERSITY

3031ICT: Applied Data Mining
3804ICT: Data Mining

Lecture 8:
Time Series and Sequential Data Mining



Dr Can Wang

School of Information and Communication Technology

Time Series and Sequential Data Mining

- Time-series Data Mining
  - Mining Time-Series Data
  - Estimation of Trend Curve
    - Free Hand Curve Method
    - Least-squared Method
    - Moving Average Method
  - Estimation of Seasonal Variations
- Sequential Data Mining
  - Sequence & Sub-sequence Definition
  - Sequential Pattern
  - Generalized Sequential Pattern (GSP)
- Summary

What is Time Series and Sequence Data Mining

Griffith UNIVERSITY

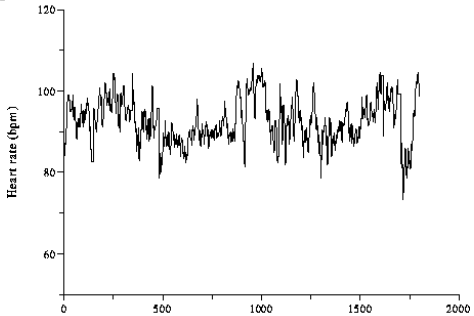
- **Time-series** Data Mining: Mining on datasets that consist of sequences of values changing with time.
- **Sequence** Data Mining: Mining on datasets that consist of sequences of events, with or without concrete notion of time.

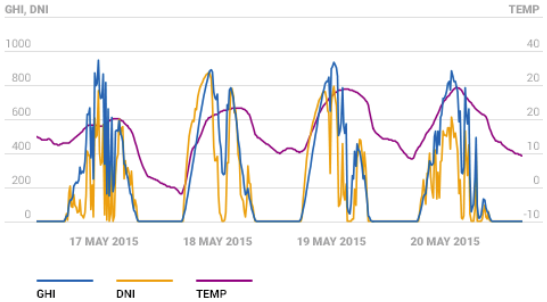
Mining Time-Series Data

- Time-series database
  - Consists of sequences of values changing with time
  - Data is recorded at **regular intervals**
  - Characteristic time-series components
    - Trend, cycle, seasonal, irregular

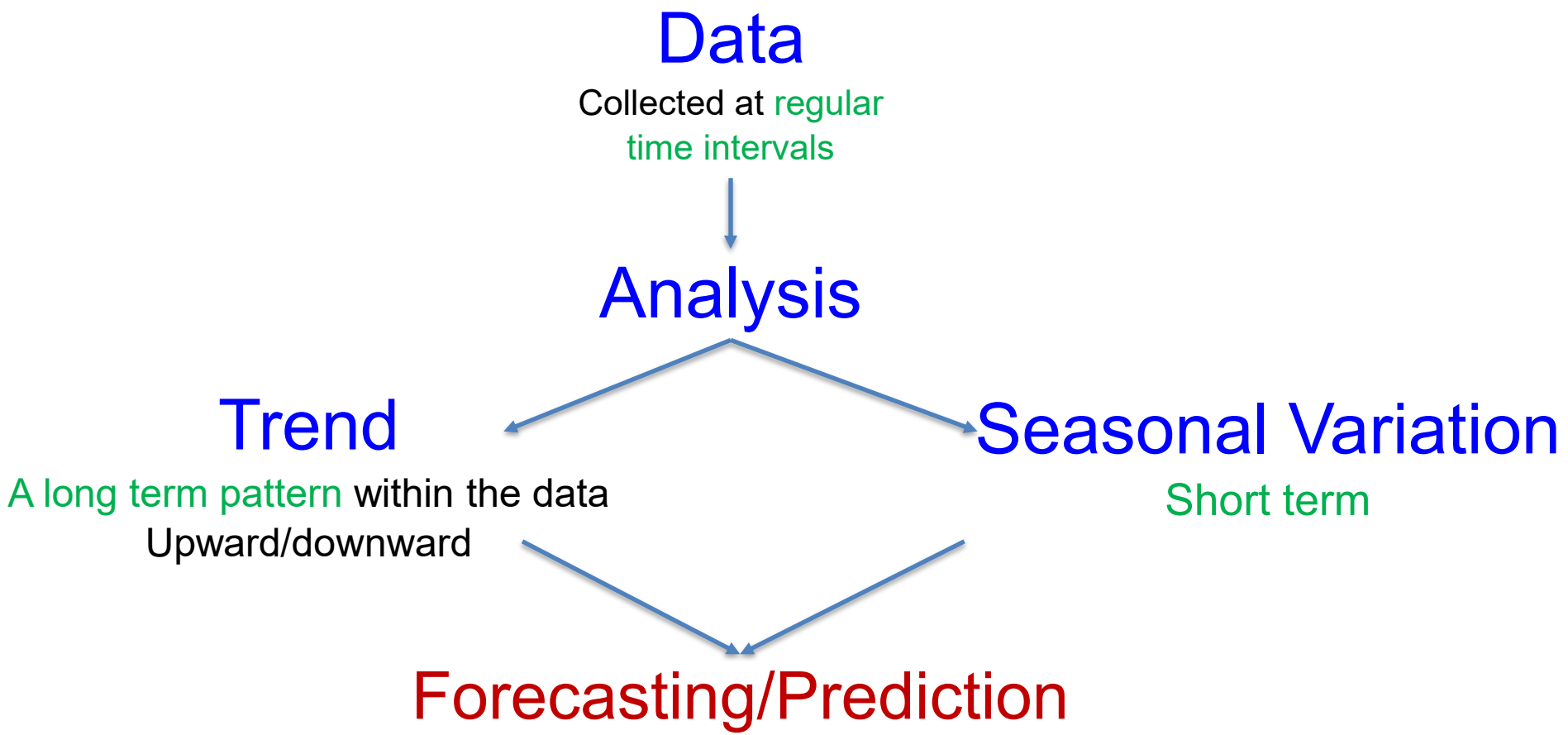


- Applications
  - Financial: stock price, inflation
  - Industrial: power consumption
  - Scientific: experiment results
  - Medical: bedside sensors



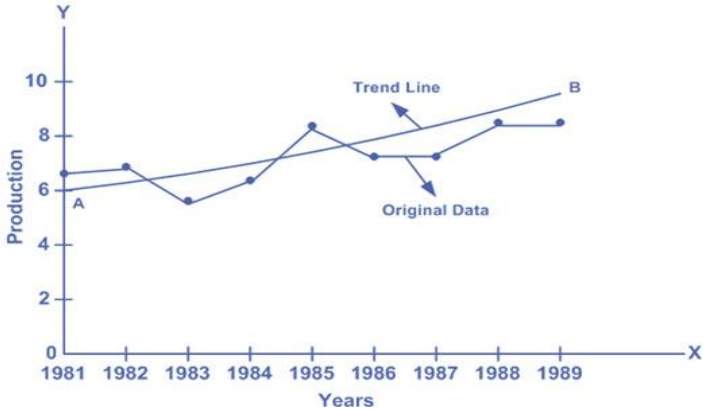


Analysis of Time Series Data



Estimation of Trend Curve

- The Free Hand Curve Method
  - Simply fit the curve by looking at the graph
  - Costly and barely reliable for large-scaled data mining



- The least-square method
  - Find the curve **minimizing the sum of the squares** of the deviation of points on the curve from the corresponding data points
- The moving-average method
  - is calculated to remove the seasonal effect, allowing us to see any trend in the data

Moving Average

- Moving average of order n

$$\frac{y_1 + y_2 + \cdots + y_n}{n}, \frac{y_2 + y_3 + \cdots + y_{n+1}}{n}, \frac{y_3 + y_4 + \cdots + y_{n+2}}{n}, \cdots$$

  - Smooth the data
  - Eliminate cyclic, seasonal and irregular movements
  - Lose the data at the beginning or end of a series
  - Sensitive to outliers (can be reduced by **weighted moving average**)

- Example

Time	1st	15th	1st	15th	1st	15th
Data	8	20	10	26	14	30
MA (n=2)	14	15	18	20	22	

- Seasonal index
  - Set of numbers showing the relative values of a variable during the months of the year
  - E.g., if the sales during October, November, and December are 80%, 120%, and 140% of the average monthly sales for the whole year, respectively, then 0.8, 1.2, and 1.4 are seasonal index numbers for these months. If we have a seasonal index = 1.0, then the data is equal to the average value of the whole year.
- Deseasonalized data
  - Data adjusted for seasonal variations for better trend and cyclic analysis
  - Divide the original monthly data by the seasonal index numbers for the corresponding months

Trend Discovery in Time-Series: Estimation of Seasonal Variations

- Seasonal data

Year	Q1	Q2	Q3	Q4
2017	76	68	67	95
2016	75	66	64	89
2015	72	64	63	75

The seasonal index

A **season index** is defined by the formula:

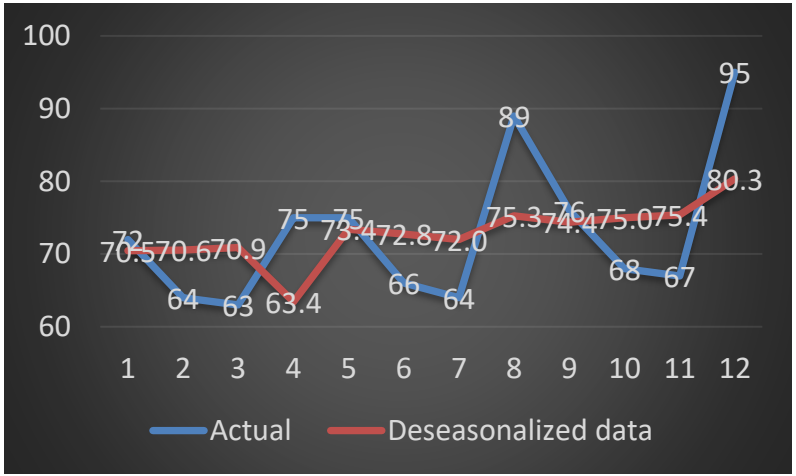
$$\text{seasonal index} = \frac{\text{value for season}}{\text{seasonal average}}$$

In this formula, the season is a month, quarter or the like.

Ave_SI_Qi=(SI_2015Qi+SI_2016Qi+SI_2017Qi)/3

Deseasonalized=Actual/Ave_SI_Qi (i=1, 2, 3, 4)

Quarter	Actual	Seasonal Index	Deseasonalized Data
2015Q1	72	1.051094891	70.5
2015Q2	64	0.934306569	70.6
2015Q3	63	0.919708029	70.9
2015Q4	75	1.094890511	63.4
2016Q1	75	1.020408163	73.4
2016Q2	66	0.897959184	72.8
2016Q3	64	0.870748299	72.0
2016Q4	89	1.210884354	75.3
2017Q1	76	0.993464052	74.4
2017Q2	68	0.888888889	75.0
2017Q3	67	0.875816993	75.4
2017Q4	95	1.241830065	80.3



Time Series and Sequential Data Mining

- Time-series Data Mining
  - Mining Time-Series Data
  - Estimation of Trend Curve
    - Free Hand Curve Method
    - Least-squared Method
    - Moving Average Method
  - Estimation of Seasonal Variations
- Sequential Data Mining
  - Sequence & Sub-sequence Definition
  - Sequential Pattern
  - Generalized Sequential Pattern (GSP)
- Summary

10

What is Time Series and Sequence Data Mining

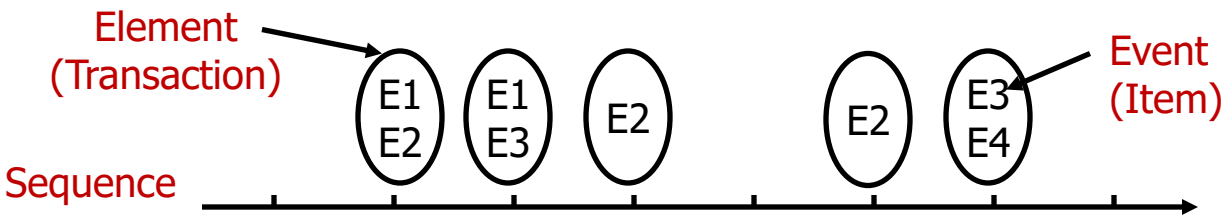
Griffith UNIVERSITY

- **Time-series** Data Mining: Mining on datasets that consist of sequences of values changing with time.

- **Sequence** Data Mining: Mining on datasets that consist of sequences of events, with or without concrete notion of time.

Sequence Data

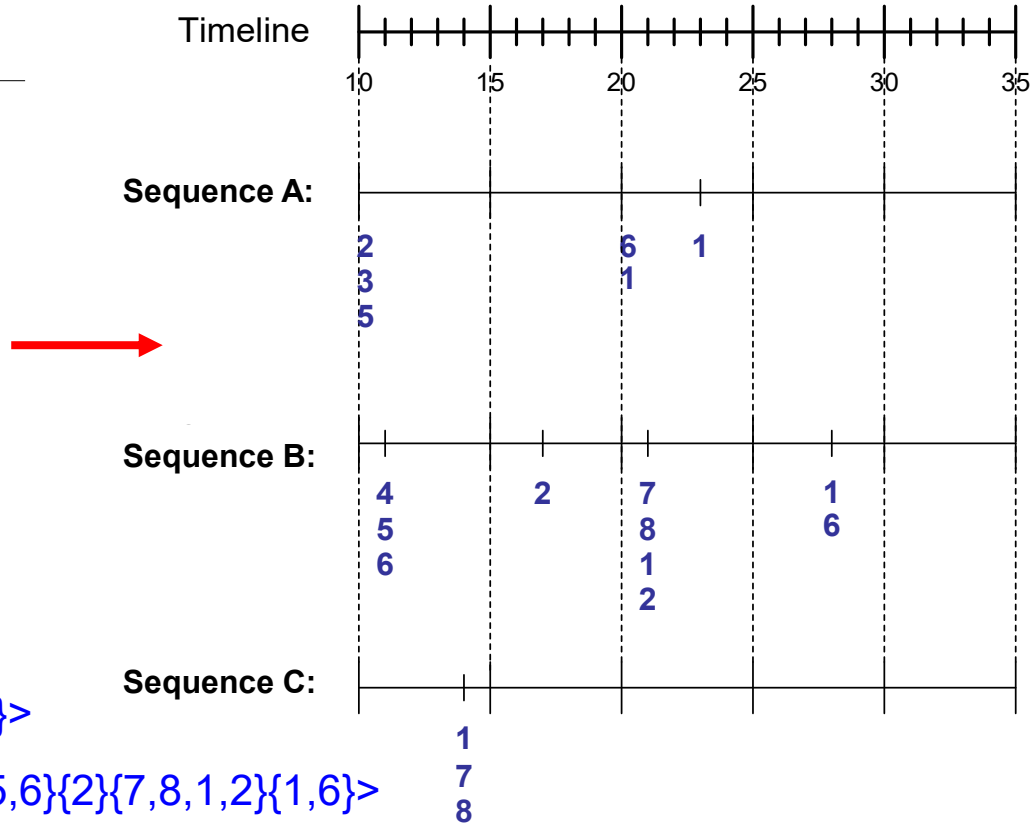
Sequence Database	Sequence	Element (Transaction)	Event (Item)
Customer	Purchase history of a given customer	A set of items bought by a customer at time t	Books, diary products, CDs, etc
Web Data	Browsing activity of a particular Web visitor	A collection of files viewed by a Web visitor after a single mouse click	Home page, index page, contact info, etc
Event data	History of events generated by a given sensor	Events triggered by a sensor at time t	Types of alarms generated by sensors
Genome sequences	DNA sequence of a particular species	An element of the DNA sequence	Bases A,T,G,C



Sequence Data

Sequence Database:

Sequence ID	Timestamp	Events
A	10	2, 3, 5
A	20	6, 1
A	23	1
B	11	4, 5, 6
B	17	2
B	21	7, 8, 1, 2
B	28	1, 6
C	14	1, 8, 7



Sequence Expression:

A: $<\{2,3,5\}_{10}\{6,1\}_{20}\{1\}_{23}>$ or $<\{2,3,5\}\{6,1\}\{1\}>$

B: $<\{4,5,6\}_{11}\{2\}_{17}\{7,8,1,2\}_{21}\{1,6\}_{28}>$ or $<\{4,5,6\}\{2\}\{7,8,1,2\}\{1,6\}>$

C: $<\{1,8,7\}_{14}>$ or $<\{1,8,7\}>$

No order within the element

Formal Definition of a Sequence

- A sequence is an ordered list of elements

$$s = < e_1\ e_2\ e_3\ \ldots >$$

  - Each element contains a collection of events (items)

$$e_i = \{i_1, i_2, \ldots, i_k\}$$

- Length of a sequence, |s|, is given by the number of elements in the sequence.
- A k-sequence is a sequence that contains ***k*** events (items)
- A k-element sequence is a sequence that contains k elements.
- E.g. *s = <{I1, I2} {I3}>:* length(s) = 2, and s is a 3-sequence.
- Events (items) within an element are unordered and we list them alphabetically. <{a,b}> = <{b,a}>
- If an item occurs multiple times in different elements of a sequence, each occurrence contributes to the value of k. E.g. *s = <{I1, I2} {I1}>:* length(s) = 2, and s is a 3-sequence.
- E.g. s= <{I1} {I1}>, s is a 2-sequence.

Formal Definition of a Subsequence

- A sequence $<a_1\ a_2 \ldots a_n>$ is contained in another sequence $<b_1\ b_2 \ldots b_m>$ $(m \geq n)$ if there exist integers $i_1 < i_2 < \ldots < i_n$ such that $a_1 \subseteq b_{i1}$, $a_2 \subseteq b_{i2}$, …, $a_n \subseteq b_{in}$

- Illustrative Example

s: b_1 b_2 b_3 b_4 b_5

t: a_1 a_2 a_3

t is a subsequence of s, if $a_1 \subseteq b_2$, $a_2 \subseteq b_3$, $a_3 \subseteq b_5$

Data sequence	Subsequence	Contain?
< {2,4} {3,5,6} {8} >	< {2} {8} >	Yes
< {1,2} {3,4} >	< {1} {2} >	No
< {2,4} {2,4} {2,5} >	< {2} {4} >	Yes
<{2,4} {2,5}, {4,5}>	< {2} {4} {5} >	No
<{2,4} {2,5}, {4,5}>	< {2} {5} {5} >	Yes
<{2,4} {2,5}, {4,5}>	< {2, 4, 5} >	No

Sequential Pattern Mining: Definition

- The support of a subsequence w is defined as the fraction of data sequences that contain w
- **A *sequential pattern* is a frequent subsequence** if the support of a subsequence is ≥ *minsup*
- **Given:**
  - a database of sequences
  - a user-specified minimum support threshold, *minsup*
- **Task:**
  - Find all subsequences with support ≥ *minsup*

Apriori property for sequences

- Let D be a database that contains a collection of data sequences d. The support of a subsequence t is the fraction of all data sequences that contain t:

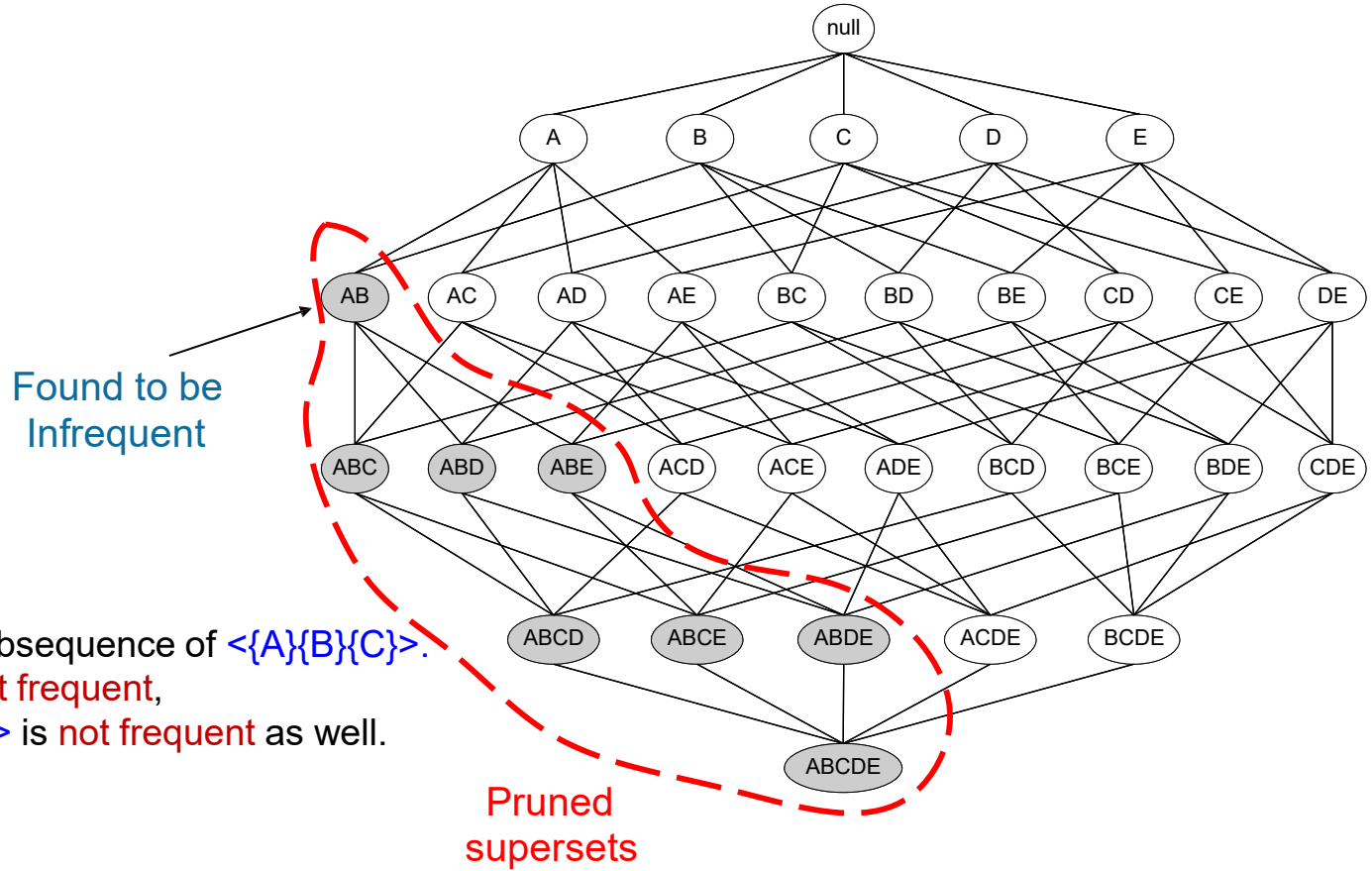
$$s(t) = \frac{|d \in D : t \; is \; a \; subsequence \; of \; d|}{|D|}$$

- Apriori property:

  If a data sequence d contains a sequence t, then it will also contain any subsequence of t.

- **Therefore:** If w is a subsequence of t, then s(w) ≥ s(t).

Illustration of Apriori Principle



<{A}{B}> is a subsequence of <{A}{B}{C}>.
If <{A}{B}> is not frequent,
then <{A}{B}{C}> is not frequent as well.

Extracting Sequential Patterns

Griffith UNIVERSITY

Mining sequential patterns algorithm: GSP (Generalized Sequential Patterns) algorithm. GSP works in almost the same way as the Apriori algorithm.

- use F_k to store the set of all frequent *k*-sequences;
- use C_k to store the set of all candidate *k*-sequences;

Initialize k=1

- C_1 -> F_1
- LOOP until F_k is empty
  - Find C_k
  - Find F_k
  - k=k+1

Brute Force Method to Extract Sequential Patterns

- Given n events: $i_1, i_2, i_3, \ldots, i_n$
- Candidate 1-subsequences:

  $\langle\{i_1\}\rangle, \langle\{i_2\}\rangle, \langle\{i_3\}\rangle, \ldots, \langle\{i_n\}\rangle$
- Candidate 2-subsequences:

  $\langle\{i_1, i_2\}\rangle, \langle\{i_1, i_3\}\rangle, \ldots,$

  $\langle\{i_1\}\ \{i_1\}\rangle, \langle\{i_1\}\ \{i_2\}\rangle, \ldots, \langle\{i_n\}\ \{i_n\}\rangle$
- Candidate 3-subsequences:

  $\langle\{i_1, i_2, i_3\}\rangle, \langle\{i_1, i_2, i_4\}\rangle, \ldots,$

  $\langle\{i_1, i_2\}\ \{i_1\}\rangle, \langle\{i_1, i_2\}\ \{i_2\}\rangle, \ldots,$

  $\langle\{i_1\}\ \{i_1, i_2\}\rangle, \langle\{i_1\}\ \{i_1, i_3\}\rangle, \ldots,$

  $\langle\{i_1\}\ \{i_1\}\ \{i_1\}\rangle, \langle\{i_1\}\ \{i_1\}\ \{i_2\}\rangle, \ldots$

If an item occurs multiple times in different elements of a sequence, each occurrence contributes to the value of *k*.

Generalized Sequential Pattern (GSP)

- **Step 1**:
  - Make the first pass over the sequence database D to yield all the frequent 1-sequences
- **Step 2**:

  Repeat until no new frequent sequences are found
  - **Candidate Generation**:
    - Merge pairs of frequent subsequences found in the (k-1)*th* pass to generate candidate sequences that contain k items
  - **Candidate Pruning**:
    - Prune candidate *k*-sequences that contain infrequent *(k-1)*-subsequences
  - **Support Counting**:
    - Make a new pass over the sequence database D to find the support for these candidate sequences
  - **Candidate Elimination**:
    - Eliminate candidate *k*-sequences whose actual support is less than *minsup*

Candidate Generation

- Base case (k=2):
  - Merging two frequent 1-sequences $<\{i_1\}>$ and $<\{i_2\}>$ will produce the following candidate 2-sequences: $<\{i_1\}\ \{i_1\}>$, $<\{i_1\}\ \{i_2\}>$, $<\{i_2\}\ \{i_2\}>$, $<\{i_2\}\ \{i_1\}>$ and $<\{i_1\ i_2\}>$.
  - Each frequent 1-sequence $<\{i_m\}>$ produces the candidate 2-sequence $<\{i_m\}\ \{i_m\}>$.

- For example, if <{A}> and <{B}> are frequent, this produces the candidate 2-sequences:
  - <{A} {B}>,<{B} {A}>, <{A,B}>, <{A} {A}> and <{B} {B}>.

Candidate Generation

- General case (k>2):
  - A frequent (*k-1*)-sequence w_1 is merged with another frequent (*k-1*)-sequence w_2 to produce a candidate *k*-sequence, if the subsequence obtained by removing an event from the first element in w_1 is the same as the subsequence obtained by removing an event from the last element in w_2
    - ❑ The resulting candidate after merging is given by extending the sequence w_1 as follows:

c1 ▪ If the last element of w_2 has only one event, then append it to w_1

c2 ▪ Otherwise add the event from the last element of w_2 (which is absent in the last element of w_1) to the last element of w_1

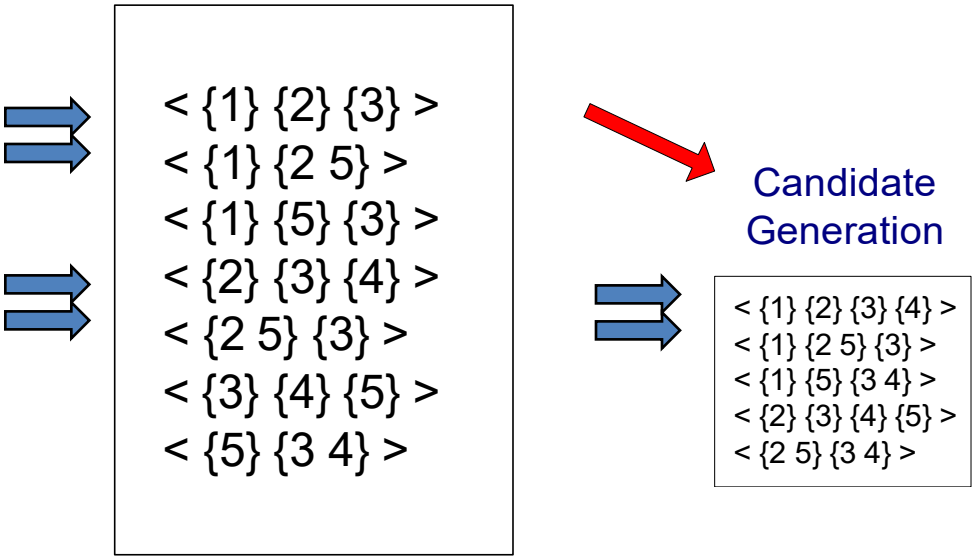
Candidate Generation Examples

- Merging w_1=<{1 2 3} {4 6}> and w_2 =<{2 3} {4 6} {5}> (<{23}{46}{x}>) c1
  produces the candidate sequence < {1 2 3} {4 6} {5}> because **the last element of w_2 has only one event**
- Merging w_1=<{1} {2 3} {4}> and w_2 =<{2 3} {4 5}> c2
  produces the candidate sequence < {1} {2 3} {4 5}> because **the last element in w_2 has more than one event**
- Merging w_1=<{1 2 3} > and w_2 =<{2 3 4} > c2
  produces the candidate sequence < {1 2 3 4}> because **the last element in w_2 has more than one event**
- We do not merge the sequences w_1 =<{1} {2 6} {4}> and w_2 =<{1} {2} {4 5}> as there is not a subsequence < {2 6}{4}{x} > in w_2

GSP Example

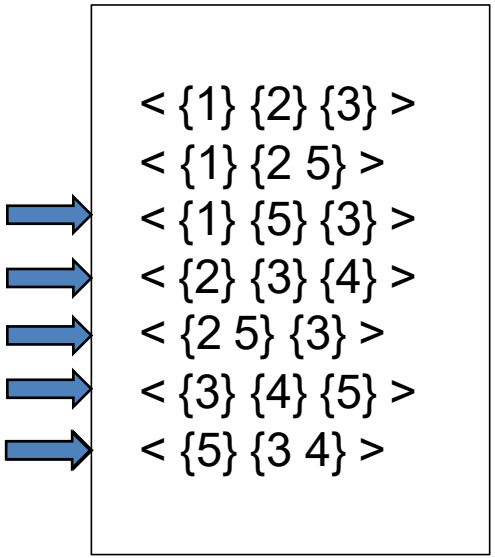
Griffith UNIVERSITY

Frequent 3-sequences



GSP Example

Frequent 3-sequences



Seq	Events
A	<{1,2,4}{2,3}{5}>
B	<{1,2}{2,3,4}>
C	<{1,2}{2,3,4}{2,4,5}>
D	<{2}{3,4}{4,5}>
E	<{1,3}{2,4,5}>

Candidate Generation

< {1} {2} {3} {4} >
< {1} {2 5} {3} >
< {1} {5} {3 4} >
< {2} {3} {4} {5} >
< {2 5} {3 4} >

<{2 5}{3 4}> is pruned because 3-subsequence <{2}{3 4}> is not frequent.

Candidate Pruning

< {1} {2 5} {3} >

A Complete GSP Example

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

C1	Support
A	4
B	4
C	4
D	5
E	4

All candidate 1-sequences are all frequent

A Complete GSP Example

Griffith UNIVERSITY

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

To generate candidate 2-sequences

	A	B	C	D	E
A	<AA>	<AB> <{AB}>	<AC> <{AC}>	<AD> <{AD}>	<AE> <{AE}>
B	<BA>	<BB>	<BC> <{BC}>	<BD> <{BD}>	<BE> <{BE}>
C	<CA>	<CB>	<CC>	<CD> <{CD}>	<CE> <{CE}>
D	<DA>	<DB>	<DC>	<DD>	<DE> <{DE}>
E	<EA>	<EB>	<EC>	<ED>	<EE>

A Complete GSP Example

Griffith UNIVERSITY

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

Scan database and find out all frequent 2-sequences

	A	B	C	D	E
A	~~<AA>~~	~~<AB>~~ ~~<{AB}>~~	<AC> ~~<{AC}>~~	<AD> ~~<{AD}>~~	<AE> ~~<{AE}>~~
B	~~<BA>~~	~~<BB>~~	<BC> ~~<{BC}>~~	<BD> ~~<{BD}>~~	<BE> ~~<{BE}>~~
C	~~<CA>~~	~~<CB>~~	~~<CC>~~	<CD> ~~<{CD}>~~	<CE> ~~<{CE}>~~
D	~~<DA>~~	~~<DB>~~	~~<DC>~~	~~<DD>~~	~~<DE>~~ <{DE}>
E	~~<EA>~~	~~<EB>~~	~~<EC>~~	~~<ED>~~	~~<EE>~~

A Complete GSP Example

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

	Frequent 2-Sequences
1	<{A} {C}>
2	<{A} {D}>
3	<{A} {E}>
4	<{B} {C}>
5	<{B} {D}>
6	<{B} {E}>
7	<{C} {D}>
8	<{C} {E}>
9	<{DE}>

Merge $F_2 * F_2$ to generate candidate C_3

1+7: <{A}{C}{D}>
1+8: <{A}{C}{E}>
2+9: <{A}{DE}>
4+7: <{B}{C}{D}>
4+8: <{B}{C}{E}>
5+9: <{B}{DE}>
7+9: <{C}{DE}>

A Complete GSP Example

Griffith UNIVERSITY

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

	Frequent 2-Sequences
1	<{A} {C}>
2	<{A} {D}>
3	<{A} {E}>
4	<{B} {C}>
5	<{B} {D}>
6	<{B} {E}>
7	<{C} {D}>
8	<{C} {E}>
9	<{DE}>

Pruning according to frequent 2-sequences

1+7: <{A}{C}{D}>
1+8: <{A}{C}{E}>
2+9: <{A}{DE}>
4+7: <{B}{C}{D}>
4+8: <{B}{C}{E}>
5+9: <{B}{DE}>
7+9: <{C}{DE}>

No one is pruned

A Complete GSP Example

Griffith UNIVERSITY

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

	Frequent 2-Sequences
1	<{A} {C}>
2	<{A} {D}>
3	<{A} {E}>
4	<{B} {C}>
5	<{B} {D}>
6	<{B} {E}>
7	<{C} {D}>
8	<{C} {E}>
9	<{DE}>

Check counts of candidate 3-sequences

1+7: <{A}{C}{D}> = 2
1+8: <{A}{C}{E}> = 2
2+9: <{A}{DE}> = 2
4+7: <{B}{C}{D}> = 2
4+8: <{B}{C}{E}> = 2
5+9: <{B}{DE}> = 2
7+9: <{C}{DE}> = 2

No one satisfies min_support (50%)

A Complete GSP Example

Griffith UNIVERSITY

Find all the frequent subsequences with support ≥ 50%, given the sequence database shown in the following table.

	Sequence Data
1	<{AB}{C}{DE}{C}>
2	<{AB}{CD}{E}>
3	<{B}{A}{B}{DE}>
4	<{C}{DE}{C}{E}>
5	<{B}{A}{BC}{AD}>

Thus, all frequent k-sequential patterns (k>1) are

<{A} {C}>
<{A} {D}>
<{A} {E}>
<{B} {C}>
<{B} {D}>
<{B} {E}>
<{C} {D}>
<{C} {E}>
<{DE}>

Time Series and Sequential Data Mining

Griffith UNIVERSITY

- Time-series Data Mining
  - Mining Time-Series Data
  - Estimation of Trend Curve
    - Free Hand Curve Method
    - Least-squared Method
    - Moving Average Method
  - Estimation of Seasonal Variations
- Sequential Data Mining
  - Sequence & Sub-sequence Definition
  - Sequential Pattern
  - Generalized Sequential Pattern (GSP)
- Summary

34

Have a question?

